

Server-Side Template Injection (SSTI)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Muchas apps meten tu entrada dentro de una plantilla (Jinja2, Twig, Freemarker...) para generar HTML dinámico. Si el motor de plantillas EVALUA tu entrada en vez de solo insertarla como texto, puedes inyectar sintaxis de la propia plantilla. Según el motor, eso escala desde leer variables internas hasta ejecutar comandos del sistema.

El programador hace algo como `render_template_string('Hola ' + nombre)` en vez de pasar nombre como variable separada. El motor no distingue 'tu texto' de 'código de plantilla que debo evaluar', igual que en `sqli` o `command-injection` la app no distingue dato de instrucción.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando tu entrada aparece reflejada en la página y el backend usa un motor de plantillas (Jinja2/Python, Twig/PHP, Freemarker o Velocity/Java, ERB/Ruby). Sospecha en campos de nombre, mensajes, plantillas de email o generación de PDF/HTML.

Cómo identificarla

- El campo se refleja en la respuesta tal cual lo mandas.
- Mensajes de error que mencionan Jinja2, Twig, Freemarker, Velocity o un motor de plantillas concreto.
- `{{7*7}}` (o el payload equivalente del motor) se convierte en 49 en la respuesta.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda una operación matemática como payload (`{{7*7}}`, `${7*7}` o `<%= 7*7 %>` según sospeches el motor) en el campo reflejado.

Evidencia esperada: La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor está EVALUANDO tu entrada, no solo mostrándola.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/?name={{7*7}}"
```

Objetivo y contexto: Prueba de confirmacion minima para Jinja2/Twig-style. Si el motor evalua, 49 aparece en la respuesta; si no, veras el texto literal.

Resultado esperado: Un '49' en la respuesta HTML confirma SSTI. Si no cambia nada, prueba otras sintaxis: `{{7*7}}` (Freemarker/Velocity) o `<%= 7*7 %>` (ERB).

Capa 2: prueba documentada

```
curl -sS "$URL/?name={{config.__class__.__init__.__globals__['os'].popen('id').read()}}"
```

Objetivo y contexto: Payload de escalada especifico de Jinja2/Flask: navega desde un objeto de config accesible hasta el modulo os para ejecutar comandos.

Resultado esperado: La salida de id (uid=... gid=...) en la respuesta confirma RCE completo, no solo evaluacion de expresiones.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/?name={{7*7}}"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor esta EVALUANDO tu entrada, no solo mostrandola.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma con una operacion matematica minima (`{{7*7}}`) antes de nada mas.
- Identifica el motor exacto: cada uno tiene payloads de deteccion distintos (Jinja2 vs Twig vs Freemarker dan resultados diferentes a la misma sintaxis).

- Una vez identificado, escala con el payload de RCE específico del motor (en Jinja2, via `__class__.__init__.__globals__` hasta llegar a `os.popen`).
- Con ejecución de comandos, confirma con `id` y salta a conseguir shell: mira `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Tu entrada se refleja pero SIN evaluar (aparece literal `{{7*7}}` en el HTML): es solo XSS/reflexión, no SSTI.
- La app no usa motor de plantillas (renderiza HTML estático o via un framework que sanea siempre).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se evalua como codigo de plantilla en vez de mostrarse como texto; segun el motor, eso llega a ejecucion de comandos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ssti\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.